

Modeling galaxy velocities with a Dirichlet process mixture model

Mattias Villani

Table of contents

Introduction	1
Dirichlet process prior for the cdf	2
Dirichlet process mixture	8

Introduction

The **galaxy velocity data**¹ consists of the velocities of 82 galaxies (measured in 1000km/s, after transformation). The scientific question is whether or not the velocities of the galaxies cluster around a few values, which would indicate that the galaxies are part of a few superclusters².

The analysis here first explores the **Dirichlet process** as a prior directly on the distribution (cdf) of the velocities before moving on to the more interesting **Dirichlet process mixture (DPM) model** with Gaussian mixture components. The DP mixture makes it possible to learn about the number of mixture components in the data, i.e. to explore the possibility of superclusters of galaxies.

```
using Pkg
Pkg.activate(joinpath(@__DIR__, ".."))
using CSV, HTTP, DataFrames, Plots, LaTeXStrings, Random
using Distributions, Measures, KernelDensity

colors = ["#6C8EBF", "#c0a34d", "#780000", "#007878",
          "#b5c6df", "#eadaaa", "#AE6666", "#4CA0A0", "#bf9d6c"]
gr(grid = false, lw = 1, titlefontsize = 12)
Random.seed!(123); # set the random seed for reproducibility
```

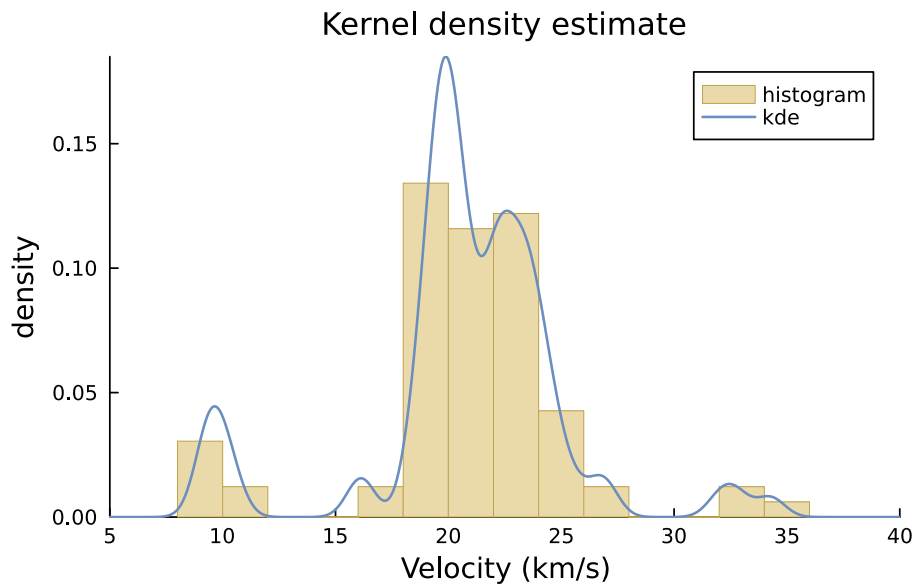
The data is available in the **galaxy** dataset in the **MASS** package in R. We inspect the data by plotting a histogram and a kernel density estimate using cross-validation to select the optimal bandwidth. At least three clusters are

¹Postman, M., Huchra, J. P. and Geller, M. J. (1986) Probes of large-scale structures in the Corona Borealis region. *Astronomical Journal* **92**, 1238–1247.

²Roeder, K. (1990). *Density estimation with confidence sets exemplified by superclusters and voids in the galaxies*. *Journal of the American Statistical Association*, 85(411), 617-624

visible in the histogram, while the kernel density estimate suggest up to seven clusters.

```
url = "https://raw.githubusercontent.com/vincentarelbundock/Rdatasets/master/csv" *
      "/MASS/galaxies.csv"
df = CSV.read(HTTP.get(url).body, DataFrame)
velocity = df.dat/1000
xlimits = [5,40]
xGrid = range(xlimits[1], xlimits[2], length = 1000)
n = length(velocity)
plt_data = histogram(velocity, xlim = xlimits, fillcolor=colors[6], linecolor = colors[2],
  normalize = true, label="histogram", lw = 0.5, xlabel="Velocity (km/s)",
  ylabel="density", title="Histogram")
kde = kde_lscv(velocity)
plot!(plt_data, xGrid, pdf.(kde, xGrid), xlim = xlimits, color=colors[1], label="kde",
  lw = 1.5, xlabel="Velocity (km/s)", ylabel="density", title="Kernel density estimate")
```



Dirichlet process prior for the cdf

We first analyze the cumulative distribution function (cdf) of the velocities using a Dirichlet process prior.

Empirical cdf

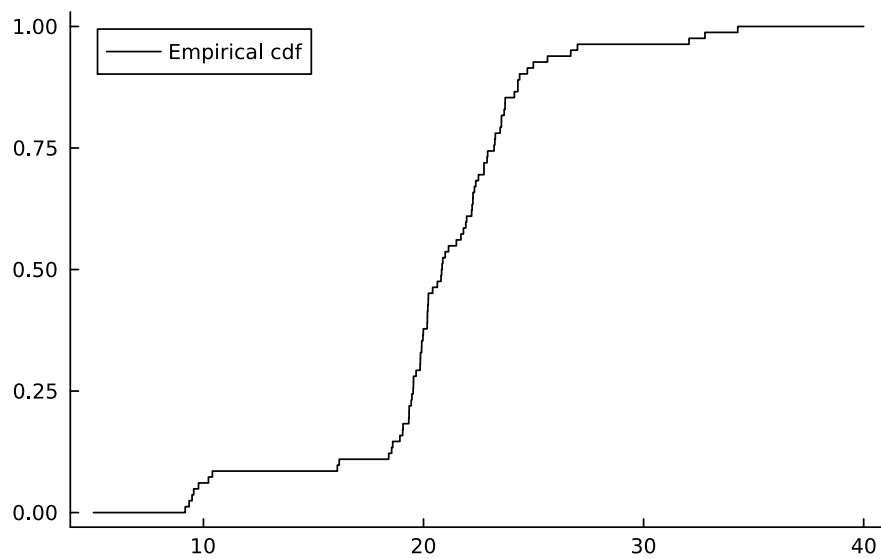
The empirical cumulative distribution function (cdf) for the data can be computed by

```

function empirical_cdf(x, xlimits)
    xs = sort(x)
    ps = (1:length(xs)) ./ length(xs)
    xs = [xlimits[1];xs;xlimits[2]]
    ps = [0;ps;1]
    return xs, ps
end

xs, ps = empirical_cdf(velocity, xlimits)
plot(xs, ps, seriestype = :steppost, color = :black, label = "Empirical cdf")

```

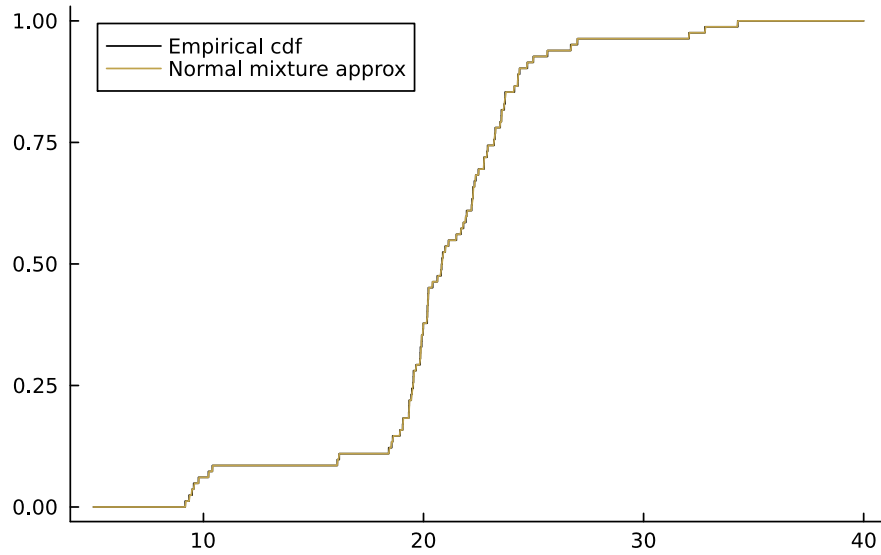


When we later combine the empirical cdf with a Dirichlet process prior to form the posterior distribution it will be convenient to have proper Julia distribution object for the empirical cdf. We therefore approximate it with a mixture of normals with components with tiny (10^{-15}) variances, effectively making it a mixture of Dirac spikes at the data points with equal weights $1/n$. The plot below shows that the approximation is virtually identical to the true empirical cdf.

```

F = MixtureModel([Normal( , 1.0e-15) for in velocity])
plot!(xGrid, cdf(F, xGrid), color = colors[2], label = "Normal mixture approx")

```



Prior distribution

Let P denote the unknown distribution for the galaxy velocities. We use a Dirichlet process for P , denoted by $P \sim \text{DP}(\alpha_0 P_0)$. The P_0 is the so called **base measure** and is our best prior guess (prior expected value) of P . We take P_0 to be the $N(30, 3^2)$ distribution. The scalar parameter α_0 is the prior concentration or **prior precision**, which is inversely related to the variance around P_0 . We start out with low prior concentration $\alpha_0 = 1$ and later explore the effect of changing α_0 .

```
P = Normal(20, 3)
    = 1
```

1

At this stage we would like to simulate draws from the DP prior and plot the resulting curves, to get a feeling for the effect of α_0 . There is a nice so called stick-breaking representation of the Dirichlet process which is constructive and makes it easy to simulate from a DP. The **stick-breaking representation** of the Dirichlet process is a mixture of point masses δ_{θ_k} at locations θ_k with weights ω_k :

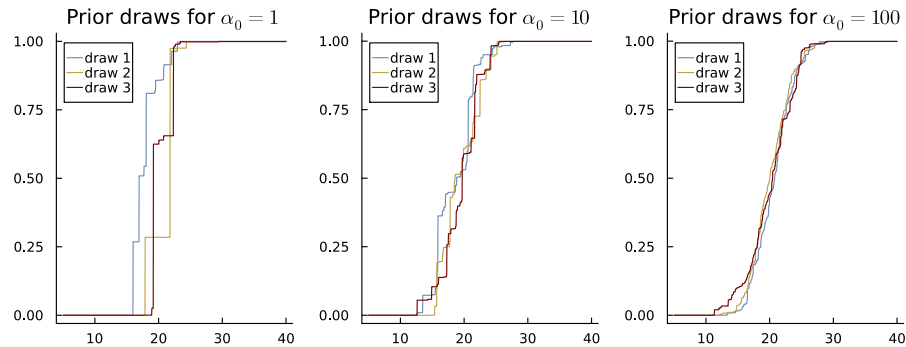
$$\begin{aligned}
P(\cdot) &= \sum_{k=1}^{\infty} \omega_k \delta_{\theta_k} \\
\omega_k &= V_k \prod_{\ell < k} (1 - V_\ell) \\
V_k &\stackrel{\text{iid}}{\sim} \text{Beta}(1, \alpha_0) \\
\theta_k &\stackrel{\text{iid}}{\sim} P_0
\end{aligned} \tag{1}$$

The function below simulates from the Dirichlet process using the stick-breaking construction. The input ϵ is the remaining stick length when the simulation terminates.

```
function SimDirProcess(P, , = 0.001)
    stickLeft = 1
    = Float64[]
    = Float64[]
    while stickLeft >
        = rand(P)
        V = rand(Beta(1, ))
        = V * stickLeft
        stickLeft = stickLeft * (1 - V)
        push!( , )
        push!( , )
    end
    sortIdx = sortperm( )
    return [sortIdx], [sortIdx]
end
```

Here are a few draws from the prior for three different choices of α_0 ; note the construction of the `cdfdraw` draw.

```
nSim = 3
plts = [plot() for _ in 1:3]
for (j, ) in enumerate([1,10,100])
    for i = 1:nSim
        , = SimDirProcess(P, )
        cdfdraw = [sum( [ .< ] ) for in xGrid]
        plot!(plts[j], xGrid, cdfdraw, color = colors[i], label = "draw $i",
            title = "Prior draws for "*L"\alpha_0 = %$( )")
    end
end
plot(plts..., layout = (1,3), size = (800,300))
```



We can simulate a large number of draws and construct 95% prior credible intervals for P , showing the variation in the prior. Let us first define a little helper function for computing quantiles for arrays:

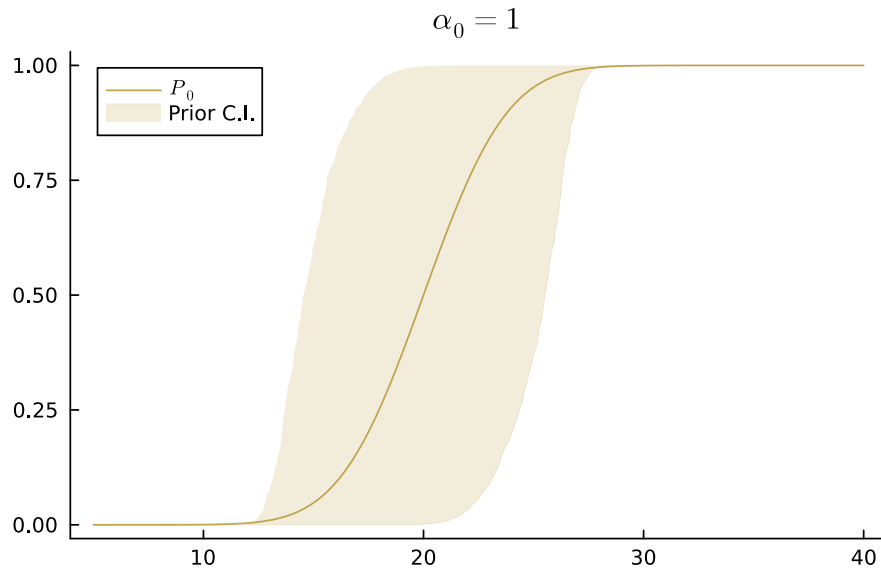
```
quantile_multidim(A, p; dims, kwargs...) = mapslices(x->quantile(x, p; kwargs...), A; dims)
```

quantile_multidim (generic function with 1 method)

Here are the (pointwise) prior credible intervals for $\alpha_0 = 1$.

```
nSim = 10000
      = 1
Priordraws = zeros(length(xGrid), nSim)
for i = 1:nSim
    , = SimDirProcess(P, )
    Priordraws[:,i] = [sum( [ .< ] ) for  in xGrid]
end
priorQuantilesCDF = quantile_multidim(Priordraws, [0.025, 0.975], dims = 2)

plot(xGrid, cdf.(P, xGrid), label = L"P_0", c = colors[2], title = L"\alpha_0 = %$( )",
     legend = :topleft)
plot!(xGrid, priorQuantilesCDF[:,1], lw = 0, fillrange = priorQuantilesCDF[:,2],
     opacity = 0.2, c = colors[2], label = "Prior C.I.")
```



Posterior distribution

The posterior distribution for P is also a Dirichlet process

$$P|x_1, \dots, x_n \sim \text{DP}(\alpha_n P_n)$$

where the posterior precision is $\alpha_n = \alpha_0 + n$ and

$$P_n = \frac{\alpha_0}{\alpha_n} P_0 + \frac{n}{\alpha_n} F_n$$

is a mixture of the prior mean and the empirical cdf. We can use the `MixtureModel` object in `Distributions.jl` package elegantly:

```
      =      + n
P = MixtureModel([P , F ], [ / , n/ ] );
```

We can now plot the prior-to-posterior mapping. Here is the code and plot for $\alpha_0 = 100$.

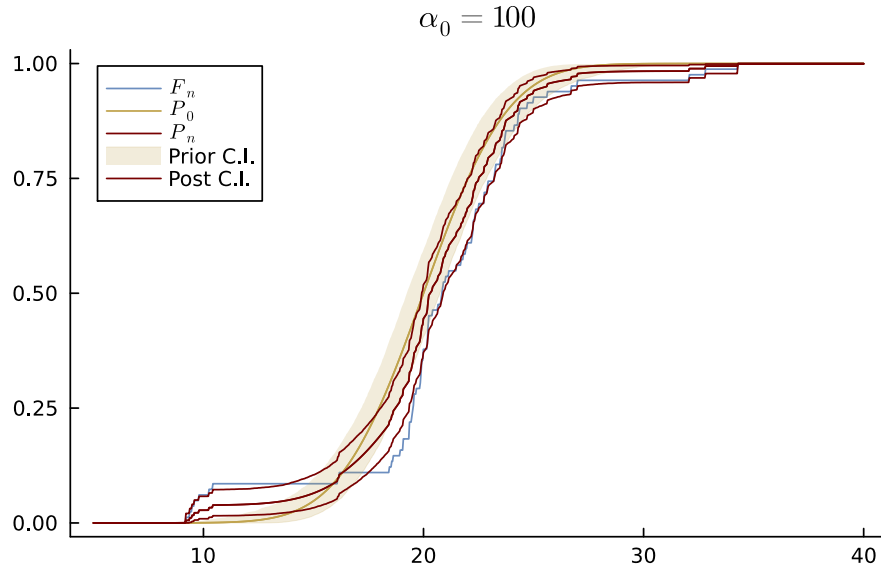
```
nSim = 5000
      = 100
      =      + n
P = MixtureModel([P , F ], [ / , n/ ] );
Priordraws = zeros(length(xGrid), nSim)
for i = 1:nSim
    , = SimDirProcess(P , );
    Priordraws[:,i] = [sum( [ .< ] ) for      in xGrid]
```

```

end
priorQuantilesCDF = quantile_multidim(Priordraws, [0.025, 0.975], dims = 2)
Postdraws = zeros(length(xGrid), nSim)
for i = 1:nSim
    , = SimDirProcess(P, );
    Postdraws[:,i] = [sum( [ .< ] ) for in xGrid]
end
postQuantilesCDF = quantile_multidim(Postdraws, [0.025, 0.975], dims = 2)

plot(xGrid, cdf.(F, xGrid), label = L"F_n", c = colors[1], title = L"\alpha_0 = %$( )",
     legend = :topleft)
plot!(xGrid, cdf.(P, xGrid), label = L"P_0", c = colors[2])
plot!(xGrid, cdf.(P, xGrid), label = L"P_n", c = colors[3])
plot!(xGrid, priorQuantilesCDF[:,1], lw = 0, fillrange = priorQuantilesCDF[:,2],
     opacity = 0.2, c = colors[2], label = "Prior C.I.")
plot!(xGrid, cdf.(P, xGrid), label = "", c = colors[2])
plot!(xGrid, postQuantilesCDF[:,1], lw = 1, c = colors[3], label = "Post C.I.")
plot!(xGrid, postQuantilesCDF[:,2], lw = 1, c = colors[3], label = "")
plot!(xGrid, cdf.(P, xGrid), label = "", c = colors[3])

```



Dirichlet process mixture

The posterior distribution for P in the above analysis is almost surely discrete. This is an undesirable consequence of the Dirichlet process prior whenever P is thought to be a continuous distribution.

However, the almost sure discreteness of the DP can be turned into a useful feature if the DP is used as mixing measure in a mixture model. A mixture model for the random variable X is of the form

$$p(x) = \int p(x|)dP()$$

where $p(x|\theta)$ is the kernel distribution with parameter vector and $P()$ is the mixing distribution. The notation $dP()$ comes from measure theory and $P(\cdot)$ is a probability measure. This allows us to let $P(\cdot)$ be both a discrete and continuous probability distribution, or a mixture of the two types. There are three basic types of mixture distributions:

1. **Continuous mixtures** where $P()$ is a continuous distribution with density $p(\theta)$ so we can write the more familiar integral $p(x) = \int p(x|)p()d$. Example: the student t density is a mixture of Gaussian kernels where the variance parameter of the kernel is mixed with respect to a scaled-inverse-chi-square distribution.
2. **Finite discrete mixtures**

$$p(x) = \sum_{k=1}^K \omega_k \cdot N(x|\mu_k, \sigma_k^2),$$

where the kernel is again Gaussian, but the mixing distribution is a mixture of K point masses for the mean and variance of the kernel. Since the mixing distribution $P(\cdot)$ is discrete here, the integral reduces to a sum.

3. **Infinite discrete mixtures**

$$p(x) = \sum_{k=1}^{\infty} \omega_k \cdot N(x|\mu_k, \sigma_k^2),$$

where the most popular version is the Dirichlet Process Mixture (DPM) model, which is obtained when $P()$ is a *random distribution* following a Dirichlet process. The Dirichlet Process Mixture (DPM) model can be written

$$\begin{aligned} x_i | i &\stackrel{iid}{\sim} p(x_i | i) \\ i | P &\stackrel{iid}{\sim} P \\ P &\sim \text{DP}(\alpha_0 P_0) \end{aligned}$$

Since realizations from the DP is almost surely discrete there will usually be several observations sharing the same parameter value j^* . In the case with Gaussian kernel, the parameter vector is $= (\mu, \sigma^2)$ and the base measure P_0 is taken to be the conjugate prior:

$$\mu|\sigma^2 \sim N\left(\mu_0, \frac{\sigma^2}{\kappa_0}\right)$$

$$\sigma^2 \sim \text{Inv} - \chi^2(\nu_0, \sigma_0^2)$$

In the Gibbs sampler presented below, we will need to compute the posterior of (μ, σ^2) for all the observations currently allocated to a mixture component. This little help function computes the posterior parameters $\mu_n, \kappa_n, \sigma_n^2$ and ν_n from a given data vector $\mathbf{x}$:

```
function PostNormalModel(x, mu0, kappa0, sigma0^2, nu0)
    if isempty(x) # no data, drawing from the prior
        return mu0, kappa0, sigma0^2, nu0
    end
    x_bar = mean(x)
    n = length(x)
    w = 1 / (1 + n)
    mu_n = w * x_bar + (1-w) * mu0
    kappa_n = 1 + n
    sumsqr = sum((x .- x_bar).^2)
    sigma_n^2 = (sumsqr + (nu0 * n) * (x_bar - mu0)^2) / kappa_n
    return mu_n, kappa_n, sigma_n^2, nu0
end
```

PostNormalModel (generic function with 1 method)

The Bayesian learning book uses the Scaled inverse Chi-square distribution, which is not available in Distributions.jl, but we can easily cook our own using the InverseGamma distribution in Distributions.jl:

```
ScaledInverseChiSq(, sigma^2) = InverseGamma(/2, *sigma^2/2)
```

ScaledInverseChiSq (generic function with 1 method)

The complete blocked Gibbs sampler for the DPM model with Gaussian components is given below. The sampler was developed in (Ishwaran & James, 2001); see also (Villani, 2026). The prior precision α_0 can be fixed, or assigned a $\alpha_0 \sim \text{Gamma}(g, h)$ prior and learned from the data. If `alpha0 = nothing` then α_0 is estimated.

```
function GibbsDPMixtureOfNormals(x, K, nIter, nBurn, P, alpha0 = nothing, g = 1, h = 1)
    n = length(x)
    z = zeros{Int, n} # mixture allocation z[i] in {1,...,K}
    post = zeros{K}

    # Initial values
    mu = zeros{K}
    kappa = zeros{K}
    sigma2 = zeros{K}
    nu = zeros{K}
```

```

if K == 1
    = [mean(x)]
else
    = collect(range(extrema(x)..., length = K))
end
2 = (var(x)/K)*ones(K)
    = (1/K)*ones(K)
V = zeros(K) # storage for the stick-breaking
if isnothing( )
    estimate_ = true
    draws = zeros(nIter)
    = 1 # initial value
else
    estimate_ = false
    draws = nothing
end

# Storage
draws = zeros(nIter, K)
2draws = zeros(nIter, K)
draws = zeros(nIter, K)
K_occupied = zeros(Int8, nIter) # n. of occupied components
for j = 1:(nBurn+nIter)

    # Update component allocations
    for i = 1:n
        post = .* pdf.(Normal.( , sqrt.(2)), x[i])
        z[i] = rand(Categorical(post / sum(post)))
    end
    nInComps = [sum(z .== k) for k in 1:K]

    # Update component parameters
    for k = 1:K
        , 2, , = PostNormalModel(x[z .== k], , 2, , )
        2[k] = rand(ScaledInverseChiSq( , 2))
        [k] = rand(Normal( , sqrt(2[k]/ )))
    end

    # Update component probabilities
    stickLength = 1
    for k = 1:(K-1)
        post = 1 + nInComps[k]
        post = + sum(nInComps[k+1:end])
        V[k] = rand(Beta(post, post))
    end
end

```

```

        [k] = V[k] * stickLength
        stickLength = stickLength * (1 - V[k])
    end
    [K] = stickLength # Whatever is left

    # Update . Scale parameterization for the Gamma
    if estimate_
        = rand(Gamma(g + K + 1, 1/(h - sum(log.(1 .- V[1:K-1])))))
    end

    if j>nBurn
        draws[j-nBurn,:] =
            ^2draws[j-nBurn,:] = ^2
        draws[j-nBurn,:] =
        K_occupied[j-nBurn] = sum(nInComps .> 0)
        if estimate_
            draws[j-nBurn] =
        end
    end

end

return draws, ^2draws, draws, K_occupied, draws

end

```

Here are the choices of prior hyperparameters and upper bound of the number of mixtures components. We run the Gibbs sampler for `nBurn=1000` iterations as burn-in and then another `nIter = 10000` for posterior inference.

```

P = , ^2 , , = 20, 1^2, 0.1, 3
K = 30 # Upper bound on the number of components
nBurn = 1000
nIter = 10000

```

10000

```

= 1
draws, ^2draws, draws, K_occupied, draws = GibbsDPMixtureOfNormals(velocity, K,
nIter, nBurn, P, )

```

```

([19.624773234055024 22.828493445693358 ... 21.256701930176234 16.712848693748377; 19.9082432
8 6.212958252623787e-9; 0.334452228190269 0.3345473754845547 ... 2.9580342391840593e-
13 1.1845944835647705e-12; ... ; 0.32144388007593305 0.38906471191003184 ... 3.1931153271882655e-
13 7.220759629116134e-15; 0.30576722307590526 0.421470964215043 ... 1.7442760746687877e-
11 1.3250350324880402e-11], Int8[8, 9, 9, 10, 9, 9, 9, 9, 9, 8 ... 7, 9, 7, 8, 8, 8, 8, 8, 7

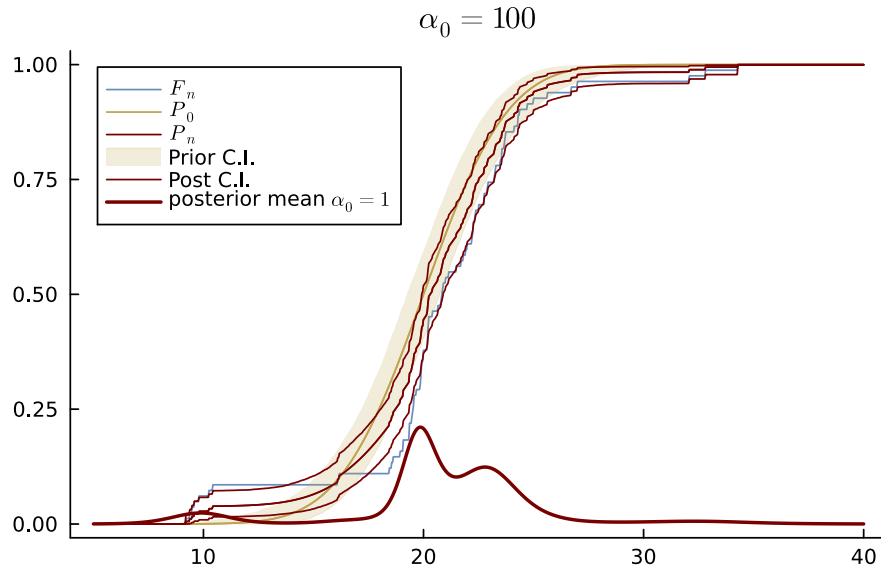
```

Plot the posterior mean of the fitted density for $\alpha_0 = 1$.

```

plt_post = plt_data
plt_iter = 1:10:nIter
pdfDraws = zeros(length(plt_iter), length(xGrid))
for (count,i) in enumerate(plt_iter)
    pdfDraws[count,:] = pdf.(MixtureModel(
        [Normal(draws[i,k], sqrt( 2draws[i,k])) for k in 1:K], draws[i,:]), xGrid)
end
postMeanPDF = mean(pdfDraws, dims = 1)[: ]
plot!(xGrid, postMeanPDF, lw = 2, c = colors[3],
    label = "posterior mean "*L"\alpha_0=%$( )")

```

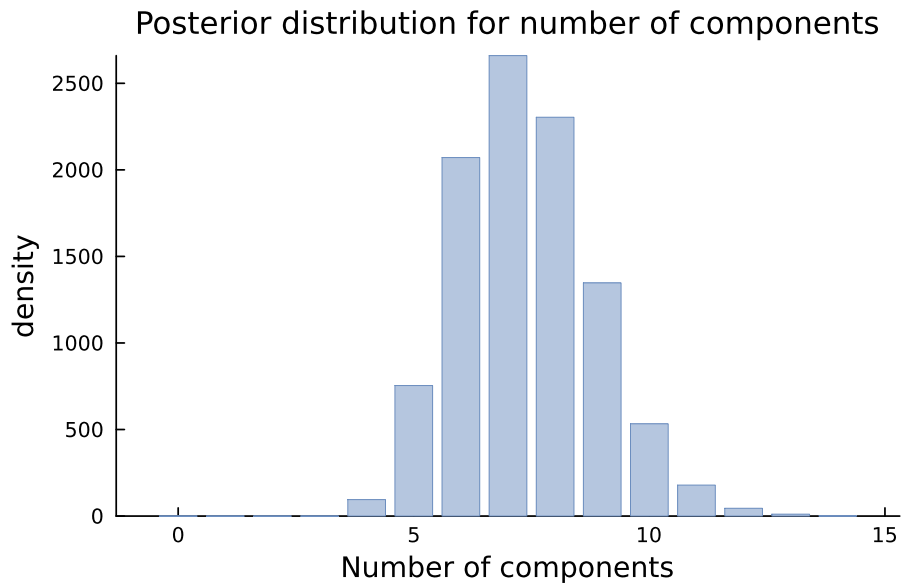


The posterior for the number of occupied mixture components has most of its mass on 5-10 components, with a mode at seven components:

```

K_max = maximum(K_occupied)
plt_bar = bar(0:K_max, [sum(K_occupied .== k) for k in 0:K_max],
    fillcolor=colors[5], linecolor = colors[1],
    normalize = true, lw = 0.5, legend = false,
    xlabel="Number of components", ylabel="density",
    title="Posterior distribution for number of components")

```



For $\alpha_0 = 10$ we get

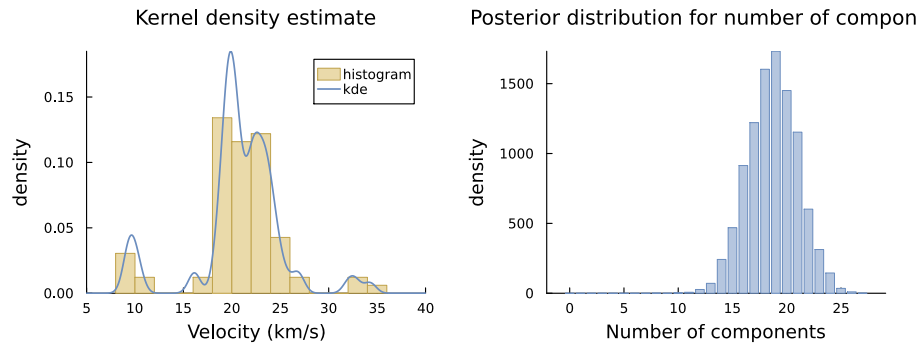
```

= 10
draws, 2draws, draws, K_occupied, draws = GibbsDPMixtureOfNormals(velocity, K,
    nIter, nBurn, P, )
plt_post = plt_data
plt_iter = 1:10:nIter
pdfDraws = zeros(length(plt_iter), length(xGrid))
for (count,i) in enumerate(plt_iter)
    pdfDraws[count,:] = pdf.(MixtureModel(
        [Normal(draws[i,k], sqrt(2draws[i,k])) for k in 1:K], draws[i,:]), xGrid)
end
postMeanPDF = mean(pdfDraws, dims = 1)[: ]
plot!(xGrid, postMeanPDF, lw = 2, c = colors[3],
    label = "posterior mean "*L"\alpha_0=%$( )")

K_max = maximum(K_occupied)
plt_bar = bar(0:K_max, [sum(K_occupied .== k) for k in 0:K_max],
    fillcolor=colors[5], linecolor = colors[1], normalize = true, lw = 0.5,
    legend = false, xlabel="Number of components", ylabel="density",
    title="Posterior distribution for number of components")

plot(plt_post, plt_bar, layout = (1,2), size = (800,300), margin = 5mm)

```



For $\alpha=100$

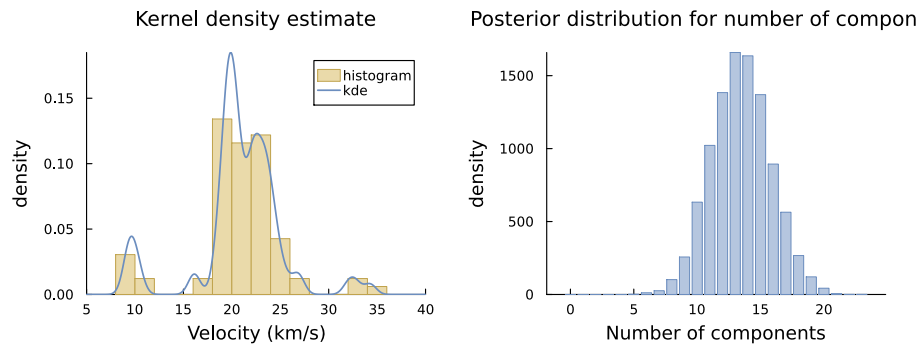
```

= 100
draws, ^draws, draws, K_occupied, draws = GibbsDPMixtureOfNormals(velocity, K,
    nIter, nBurn, P, )
plt_post = plt_data
plt_iter = 1:10:nIter
pdfDraws = zeros(length(plt_iter), length(xGrid))
for (count,i) in enumerate(plt_iter)
    pdfDraws[count,:] = pdf.(MixtureModel(
        [Normal(draws[i,k], sqrt(^draws[i,k])) for k in 1:K], draws[i,:]), xGrid)
end
postMeanPDF = mean(pdfDraws, dims = 1)[: ]
plot!(xGrid, postMeanPDF, lw = 2, c = colors[3],
    label = "posterior mean "*L"\alpha_0=%$( )")

K_max = maximum(K_occupied)
plt_bar = bar(0:K_max, [sum(K_occupied .== k) for k in 0:K_max],
    fillcolor=colors[5], linecolor = colors[1], normalize = true, lw = 0.5,
    legend = false, xlabel="Number of components", ylabel="density",
    title="Posterior distribution for number of components")

plot(plt_post, plt_bar, layout = (1,2), size = (800,300), margin = 5mm)

```

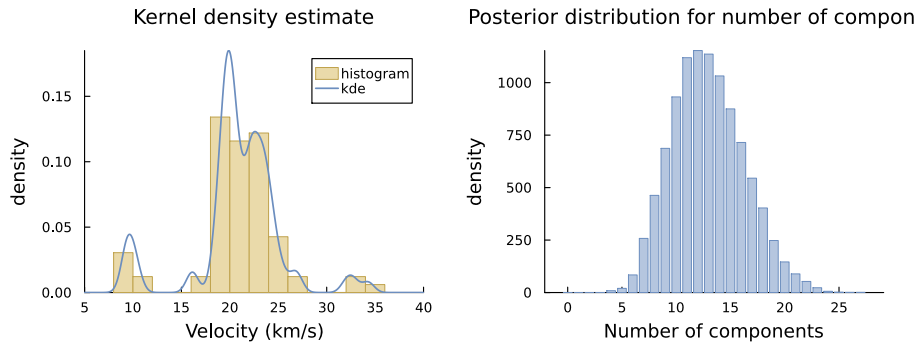


and finally when α_0 is estimate using a Gamma(1,1) prior:

```
draws, ^draws, draws, K_occupied, draws = GibbsDPMixtureOfNormals(velocity, K,
    nIter, nBurn, P)
plt_post = plt_data
plt_iter = 1:10:nIter
pdfDraws = zeros(length(plt_iter), length(xGrid))
for (count,i) in enumerate(plt_iter)
    pdfDraws[count,:] = pdf.(MixtureModel(
        [Normal(draws[i,k], sqrt(^draws[i,k])) for k in 1:K], draws[i,:]), xGrid)
end
postMeanPDF = mean(pdfDraws, dims = 1)[:]
plot!(xGrid, postMeanPDF, lw = 2, c = colors[3],
    label = "posterior mean "*L"\alpha_0=%$( )")

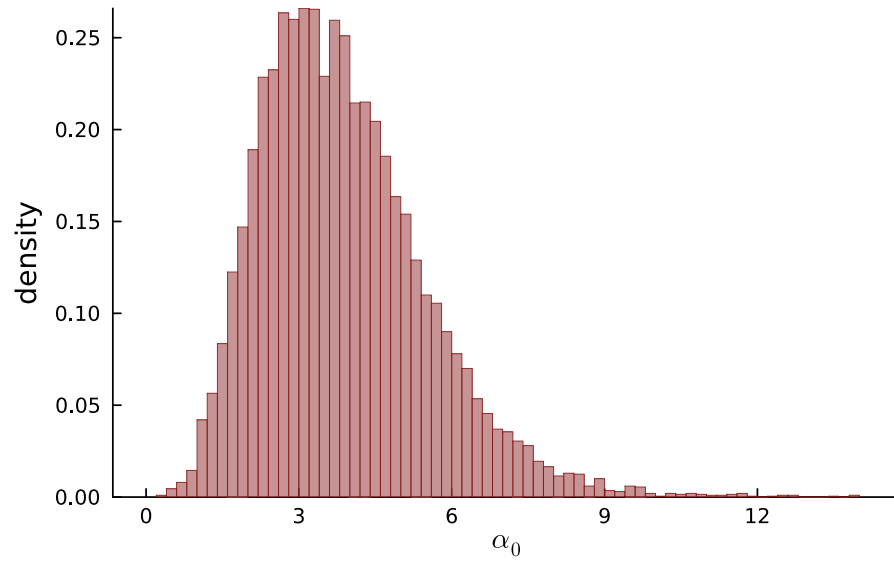
K_max = maximum(K_occupied)
plt_bar = bar(0:K_max, [sum(K_occupied .== k) for k in 0:K_max],
    fillcolor=colors[5], linecolor = colors[1], normalize = true, lw = 0.5,
    legend = false, xlabel="Number of components", ylabel="density",
    title="Posterior distribution for number of components")

plot(plt_post, plt_bar, layout = (1,2), size = (800,300), margin = 5mm)
```



The posterior for α_0 is

```
histogram( draws, fillcolor=colors[7], linecolor = colors[3], opacity = 0.7,
    normalize = true, lw = 0.5, legend = false, xlabel = L"\alpha_0", ylabel = "density")
```



Ishwaran, H., & James, L. F. (2001). Gibbs sampling methods for stick-breaking priors. *Journal of the American Statistical Association*, 96(453), 161–173.

Villani, M. (2026). *Bayesian learning - principles and computation*. <https://github.com/mattiasvillani/BayesianLearningBook/raw/main/pdf/BayesianLearningBook.pdf>